

Politecnico di Torino

Department of CONTROL AND COMPUTER ENGINEERING (DAUIN)

Master's degree in Mechatronic Engineering

2025/2026

Robotics Project

Project Report

Team 50 — Matricola:

Hussein Ghaddar — s350549

Maya Kawssan — s350682

Fatima Zouhour — s361126

Rawan El-Halabi — s361422

Note: LLMs were used mainly in code development, project knowledge, and presentations.

Role Division:

Fatima Zouhour: *Part01 Simulation + Part02 Kalman & Fusion*

Hussein Ghaddar: *Part01 Simulation (refined / reuploaded) + Video*

Rawan El-Halabi: *Part02 Kalman & Fusion*

Maya Kawssan: *Part03 Neural Network*

Note that each member was completely responsible for the development, report, presentation, and video of his part.

0. Abstract

Achieving accurate localization remains a fundamental goal in autonomous robotics, as precise control relies on accurate knowledge of the robot’s position. A core strategy in localization is the use of the Global Positioning System (GPS), which provides reliable global position estimates in outdoor environments.

However, GPS has a critical limitation: it fails in indoor scenarios where satellite signals are blocked. To address this, multiple complementary sensors can be used — namely an Inertial Measurement Unit (IMU), comprising a gyroscope and accelerometer, along with a wheel odometer — to compensate for GPS unavailability.

These sensors, however, introduce their own challenges, including noise accumulation over time and positioning errors caused by wheel slippage. Kalman filters are employed to mitigate the drift and noise accumulation inherent in raw sensor data, while an Artificial Neural Network (ANN), trained on GPS-available data, serves as a pseudo-sensor to predict the robot’s position during GPS outages. A Fuzzy Logic System (FLS) further refines the estimates by handling wheel slippage conditions.

This project is a simulation-based reproduction of the multisensor fusion framework proposed by Yousuf and Kadri (2020), and represents a successful implementation of the full pipeline — from sensor simulation to algorithmic correction — achieving accurate robot localization in both indoor and outdoor conditions.

1 Simulation

In order to simulate the experiment, we used the same simulator used in the paper, which is CoppeliaSim. In CoppeliaSim we built the scene that serves as the base for the rest of the code development. In the scene we placed a Pioneer_P3DX robot (the default robot inside CoppeliaSim) in a suitable location in the simulation region, and then added the applicable sensors to the robot frame — the GPS, accelerometer, and gyroscope (we did not find an odometer sensor) — which provide the ideal data from the simulation.

For flexibility, we decided to work using MATLAB to control the simulation and to run the algorithms. The connection between the CoppeliaSim simulator and MATLAB is provided through the remote API. Using MATLAB, we were able to start the simulation, issue commands, retrieve information and variables, and end the simulation.

Using the API, we built the base MATLAB file that receives the ideal data from the GPS, accelerometer, and gyroscope in the simulator, adds noise to the incoming data, and performs the suitable calculations, so as to emit realistic data containing pseudo-experimental conditions for the Kalman and Neural Network stages.

Moreover, due to the absence of an odometer sensor in the simulator, we modelled the odometer in MATLAB: we obtain ideal data about the robot from the simulator, build the ideal model of the odometer, then add noise and the wheel-slippage effect to the model in order to make it realistic.

The noise added to the ideal data received or calculated is zero-mean noise with the respective standard deviations mentioned in Section 4.2 of the paper. In addition, the reference trajectory was chosen to be a simple circle for simplicity, and was realized by giving different velocity commands to each wheel of the Pioneer_P3DX.

2 Kalman

When GPS is available, the goal of the Kalman stage is to fuse it with the dead-reckoning sensors to obtain a position estimate more accurate than any single sensor. GPS is noisy but does not drift, whereas the INS (accelerometer and gyroscope) and the odometer are always available but accumulate error over time — the INS through double integration of acceleration, and the odometer through single integration of velocity and wheel slippage. We therefore implemented two Extended Kalman Filters (EKF), each fusing GPS with one dead-reckoning source, and combined them using a complementary filter as in Eq. (2) of the reference paper.

EKF-1 (GPS + INS) uses a five-element state — planar position, planar velocity, and heading $[x, y, v_x, v_y, \theta]$. In the prediction step the body-frame accelerometer readings are rotated into the world frame using the current heading and integrated twice to propagate position, while the gyroscope yaw rate propagates the heading. When a GPS measurement is available it corrects the position. The filter uses the Joseph-form covariance update for numerical stability, re-symmetrises the covariance at every step, and wraps the heading to $(-\pi, \pi]$.

EKF-2 (GPS + Odometer) uses a three-element state $[x, y, \theta]$ with a unicycle motion model driven by the odometer’s forward velocity and yaw rate, again corrected by GPS when available and using the same numerical safeguards. Because odometry integrates velocity only once, EKF-2 drifts more slowly than EKF-1 when GPS is lost.

Both filters were implemented manually in base MATLAB: the prediction and update equations, the Jacobians, and the Joseph-form covariance update were coded directly rather than relying on a dedicated estimation toolbox; the only built-in routine used is `smoothdata` for Savitzky–Golay smoothing of the raw GPS.

Complementary fusion combines the two filters as a weighted sum whose weights sum to one:

$$[x_{\text{fused}}, y_{\text{fused}}] = \alpha_1 [x_{\text{KF1}}, y_{\text{KF1}}] + (1 - \alpha_1) [x_{\text{KF2}}, y_{\text{KF2}}]. \quad (1)$$

We selected $\alpha_1 = 0.1$, weighting the estimate strongly toward EKF-2.

Results. Over the full run, the fused estimate reduces the overall (Euclidean) position RMSE to 0.37 m, roughly one-third of EKF-2 alone (1.13 m). The per-axis RMSE against ground truth is given in Table 1.

Method	RMSE x (m)	RMSE y (m)
EKF-1 (GPS + INS)	1.78	0.16
EKF-2 (GPS + Odometer)	0.81	0.80
Fused ($\alpha_1 = 0.1$)	0.79	0.72

Table 1: Per-axis RMSE of the two EKFs and the complementary fusion over the full run.

Discussion and limitations. The fused output is at least as accurate as the better single filter on both axes. The most notable feature is EKF-1’s large error in x (1.78 m) compared with its small error in y (0.16 m). This asymmetry is not a filter defect: during the two GPS outages, EKF-1 dead-reckons by double-integrating acceleration, and because the robot’s velocity at those instants is directed mainly along x , the drift accumulates almost entirely in that axis. EKF-2, integrating velocity only once, drifts far less. This explains our choice to weight the fusion toward EKF-2 (small α_1), and it motivates the neural-network pseudo-sensor of Section 3, which is required to maintain accuracy through longer GPS outages where dead reckoning alone is insufficient.

3 Neural Network

When the robot is indoors, the GPS is no longer available, so our previous approach using the two Kalman-filter fusion is no longer beneficial as the estimate will drift. To solve this, we use an Artificial Neural Network (ANN) that acts as a pseudo-sensor in the indoor case. This ANN is trained when GPS is available (outdoors) so that it can predict position estimates indoors, where the final estimate is the weighted sum of KF-2 (GPS + Odometer) and the ANN prediction. To determine the weights, we used fuzzy logic, which computes them based on the difference in velocity between the odometer and the INS.

The coding of the ANN was done in MATLAB using the Deep Learning Toolbox. We built a neural network with two hidden layers. In contrast to the paper, we extended the input vector for ANN training to 13 elements, taking into consideration more data related to y (the paper’s input vector focused more on x). This change ensures that our ANN trains well on both axes.

After training the ANN, we ran the algorithm for 500 iterations with $dt = 0.05$. During this time we introduced two outages: the first from $t = 260$ to $t = 300$, and the second (long) outage from $t = 400$ to $t = 470$. The results are reported in Table 2.

Method	Outage 1				Outage 2			
	MeanX	MeanY	StdX	StdY	MeanX	MeanY	StdX	StdY
KF-2	0.1970	-0.1170	1.3661	1.2210	0.3773	0.5895	1.8455	1.8164
ANN	-0.0784	0.6671	0.2251	0.2368	-0.0424	0.8914	0.2586	0.2744
Fusion	-0.1273	0.5456	0.3223	0.3552	-0.0871	0.8036	0.4580	0.4594

Table 2: Mean and standard deviation of position error (m) during the two GPS outages for KF-2 alone, the ANN, and the ANN+KF-2 fusion.

As we can see, KF-2 drifts considerably, while the ANN and the fusion have a closer performance; however, it is noticeable that the ANN outperforms the fusion, due to the effect of KF-2 on the fused estimate. Taking the ANN prediction as the indoor estimate could be a better choice in this case, or training the ANN to have a very low error so that the fused estimate is further improved.